

SOVEREIGN

COMPLETE SYSTEM MANUAL & VIVA PRESENTATION GUIDE

Distributed Computing Simulator | Leader Election Algorithms | Lamport Clocks | Full TCP Sockets & WebSockets

---

1. Executive Summary: What is Sovereign?

Sovereign is an interactive 3D simulation of a **Distributed Computing System** disguised as a fantasy kingdom of floating sky castles.

In the real world, distributed systems power companies like **Google, Amazon, Netflix, and online banks**. Instead of running on a single computer, these systems distribute their work across dozens, hundreds, or thousands of independent server computers (called **Nodes**).

To prevent chaos (e.g., two computers modifying the same database record at the same moment), **one node must always be elected the "Leader" (or "Coordinator")**.

> **The Fundamental Problem in Distributed Computing:**
> What happens if the leader server loses power, crashes, or gets cut off from the network? The remaining living computers must independently detect the leader's death and elect a single new leader *without any human administrator intervention*.

Sovereign visually demonstrates how distributed nodes solve this problem using the two classic textbook algorithms: the **Bully Algorithm** and the **Ring Algorithm** over raw, independent TCP sockets.

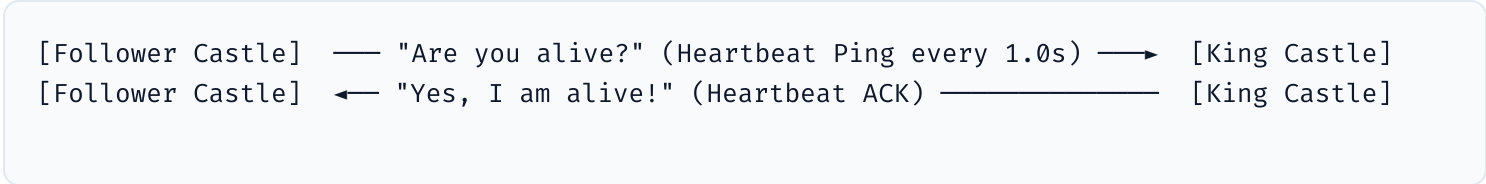
2. The Kingdom Metaphor: How to Visualize It

Every fantasy element on the screen directly maps to a technical distributed systems concept:

VISUAL IN SOVEREIGN	TECHNICAL CONCEPT IN DISTRIBUTED SYSTEMS
Floating Sky Island	The local network / cluster boundary hosting all machines.
Each Castle (1 to 5)	An **independent computer program (OS process)** running on its own local TCP port (e.g., 9001, 9002, 9003).
Castle Number	The unique **Process ID (Node ID)**. In distributed algorithms, higher IDs indicate higher rank/priority.
The Sovereign (King)	The **Elected Coordinator (Leader)** node currently in charge of the cluster.
The Golden Crown	The **Coordinator Token/Status**. Only one living node can wear the crown at any time.
Floating Sky Crown	The **Vacant State**. When the king crashes, the crown ascends to the center sky `[0, 7.2, 0]` until a new leader is elected.
Glowing Flying Orbs	Real **TCP network packets** traveling across the network between computer processes.
Tilted Broken Castle	A **Dead / Crashed Node** that cannot send or receive network traffic.
Blinking Blue Ring	An **Electing Node** actively broadcasting or processing election ballots.

3. How Castles Detect King Failure: The Heartbeat

In a distributed system, computers cannot "see" each other physically. The only way they know if another computer is alive is by sending messages over the network.



Here is the exact lifecycle of failure detection in Sovereign:

1. **Periodic Ping**: Every **1.0 second**, every living follower castle sends a tiny TCP packet to the King: *"Are you alive?"*
2. **Immediate Reply**: As long as the King process is healthy, it immediately replies with an acknowledgement: *"Yes, I am alive!"*
3. **Failure Detection**: When the King is assassinated or crashes:
   - The King stops replying to heartbeats.
   - The followers wait for **2.2 seconds** (the failure timeout threshold).
   - Once 2.2 seconds pass without any response, the follower declares: *"The King is dead!"*
4. **Crown Ascends to the Sky**: The golden crown lifts off the dead castle and hovers high above the island. Surviving castles immediately trigger an election.

4. Algorithm 1: The Bully Algorithm (Garcia-Molina, 1982)

> **The Core Motto:** *"The node with the highest ID always bullies everyone smaller."*
> Priority is determined strictly by Node ID (Process ID). Node 5 outranks Node 4, Node 4 outranks Node 3, and so on.

```
[King Node 5 Crashes]
        |
        ▼
[Node 1 notices 2.2s heartbeat timeout]
        |
        ▼
[Node 1 broadcasts ELECTION to all higher IDs: 2, 3, 4, 5]
        |
        ▼
[Nodes 2, 3, 4 reply 'OK' to Node 1: "Quiet, we are higher than you!"]
        |
        ▼
[Node 4 broadcasts ELECTION to Node 5]
        |
        ▼
[Node 5 is dead (No answer after 1.0s timeout)]
        |
        ▼
[Node 4 broadcasts COORDINATOR: "I am your new Sovereign!"]
        |
        ▼
[Golden crown swoops down and crowns Node 4]
```

The 5 Steps of Bully Election:

1. **Initiation**: Any node that notices the leader died sends an `ELECTION` packet to **all nodes with higher IDs**.
2. **The "OK" Response**: If any higher node is alive, it sends back an `OK` packet: *"I have a higher ID than you, stand down, I'll take over."* The lower node stops its election and waits.
3. **Escalation**: The higher node now sends `ELECTION` to any nodes higher than itself.
4. **Victory Claim**: If no higher node answers within the timeout (1.0 second), this node realizes it is the highest living node in the entire network! It broadcasts a `COORDINATOR` packet to all nodes.
5. **Revival Takeover (The Bully Rule)**: If the highest node (e.g. Node 5) was dead and suddenly revives (comes back to life), it does not wait politely. Because its ID is higher, it immediately deposes the current leader and

takes the crown back!

> **Mathematical Complexity:**
> • Worst-case message traffic: **$O(N^2)$**. If the lowest node detects failure, everyone messages everyone higher: N(N-1)/2 messages.
> • Very loud, chatty network traffic, but reaches leader decision quickly because broadcasts happen in parallel.

5. Algorithm 2: The Ring Algorithm (Chang & Roberts, 1979)

> **The Core Motto:** *"An orderly, polite relay race in a circle."*
> Castles are logically arranged in a circular ring: Castle 1 → Castle 2 → Castle 3 → Castle 4 → Castle 5 → Castle 1.

```
Castle 1 (detects failure, initiates list: [1])
       │
       ▼
Castle 2 (receives [1], appends own ID: [1, 2])
       │
       ▼
Castle 3 (receives [1, 2], appends own ID: [1, 2, 3])
       │
       ▼
Castle 4 (receives [1, 2, 3], appends own ID: [1, 2, 3, 4])
       │ (skips dead Castle 5 over TCP)
       ▼
Returns back to Castle 1!
Castle 1 inspects list: Winner = Max(1, 2, 3, 4) = Castle 4!
Castle 1 circulates RING_COORDINATOR token: "Castle 4 is Sovereign!"
```

The 5 Steps of Ring Election:

1. **Initiation**: When a node detects leader failure, it creates an **Election Token** containing an active list: `[1]`.
2. **Sequential Forwarding**: It forwards the token clockwise to its alive successor (Castle 2).
3. **Appending IDs**: Castle 2 appends its ID (`[1, 2]`) and forwards to Castle 3. Castle 3 appends its ID (`[1, 2, 3]`) and forwards to Castle 4.
4. **Fault Tolerance (Skipping Dead Nodes)**: Castle 4 tries to contact Castle 5. Because Castle 5 is dead (TCP connection fails), Castle 4 automatically **skips Castle 5** and connects directly to Castle 1!
5. **Coronation**: When the token completes a full circuit and returns to initiator (Castle 1), Castle 1 selects the maximum number from the list (**Max = 4**) and circulates a `RING_COORDINATOR` token declaring Castle 4 the new King.

> **Mathematical Complexity:**
> • Message traffic: **O(N)**. The token visits each node once during election and once during announcement

(roughly 2N messages).
- Highly efficient, predictable, and low network bandwidth.

6. Every Button & Control Explained in Detail

Bottom Control Dock (Main Controls)

CONTROL / BUTTON	WHAT YOU SEE IN PLAIN ENGLISH	WHAT HAPPENS BEHIND THE SCENES
💀 **Assassinate King**	Instantly strikes down the current King with a lightning bolt.	Sends `POST /api/cluster/assassinate`. Kills the leader process, sets status to `DEAD`, lifts crown to the sky, and surviving followers begin an election.
🗳️ **Hold Election**	Manually triggers an election right now.	Sends `POST /api/cluster/election/start`. Forces living castles to exchange election orbs even if no node is dead.
⚡ **Chaos Mode: ON / OFF**	Unleashes a random thunderstorm that crashes and revives random castles.	A background worker randomly kills living nodes and revives dead nodes every 4–7 seconds to prove the system's fault-tolerance in chaotic conditions.
🏰 **Nodes Slider (3–9)**	Dynamically scales the number of castles in the kingdom.	Sends `POST /api/cluster/scale`. Spawns new Python OS processes for new nodes, or cleanly terminates excess processes.
🎚️ **Sliders Icon**	Toggles the Network Fault Injection drawer.	Reveals the simulated network latency and packet drop sliders.
⏱️ **Latency Slider (0–1000ms)**	Simulates slow satellite internet or bad cables.	Instructs TCP sockets to sleep for *X* ms before processing packets. Glowing orbs travel visibly slower through the sky.
📦 **Drop Rate Slider (0–50%)**	Simulates flaky Wi-Fi or lost packets.	Causes nodes to randomly drop *X%* of incoming messages to test algorithm retry mechanisms.

Top Navigation Bar & Utilities

CONTROL	WHAT IT DOES IN PLAIN ENGLISH
BULLY / RING Selector	Instantly toggles between the Bully and Ring election algorithms without restarting the servers.
🌞 / 🌙 Day & Night Toggle	Switches between vibrant Day Mode (crystal-clear black castle numbers) and dramatic Night Mode (cinematic dark sky with glowing runes).
Heartbeat Toggle (WiFi icon)	Toggles visual rendering of the periodic blue heartbeat pings flying between followers and the King (off by default to keep the view clean).
Tab 1: 🏰 Kingdom	Main 3D interactive floating kingdom view.
Tab 2: 📊 Analytics	Opens the Performance Benchmarking and Scientific Comparison laboratory.
Tab 3: ⏱️ Replay	Opens the Lamport Logical Time Scrubber to step through past events in causal order.
Sovereign Status Badge	Shows `👑 Sovereign: Node X` or a pulsing red alert: `🔴 Crown Vacant (Electing...)`.

Left Drawer: 🏰 Castles Panel

Click the 🏰 **Castles** floating pill on the left edge of your screen to open:

- **Individual Castle Cards**: Displays Node ID, real TCP Port (e.g., `Port 9001`), and current state (`LEADER`, `ALIVE`, `ELECTING`, or `DEAD`).
- 💀 **Kill Button**: Allows you to assassinate any specific castle of your choice with one click.
- ❤️ **Revive Button**: Brings a dead castle back to life. (In Bully mode, revives trigger an instant takeover challenge!).

Right Drawer: 📜 Live Logs Panel

Click the 📜 **Live Logs** floating pill on the right edge of your screen to open:

- **Real-time Packet Stream**: Displays the exact TCP packets traveling across your computer with millisecond timestamps and Lamport clock values.
- **Filter Tabs**: Filter logs by message type: `ELECTION`, `OK`, `COORDINATOR`, or `HEARTBEAT`.
- **Trash Icon**: Clears the on-screen logs.

7. What the "Analytics" Feature is Doing (Tab 2)

Clicking **Analytics** opens the **Distributed Systems Science Laboratory**. Its purpose is to give you undeniable empirical evidence to show your professors and examiners that the theoretical math matches actual system execution!

1. **Message Complexity Chart (Total Packets Sent)**:
   - **Bully (Orange)**: Quadratic curve $O(N^2)$. As the cluster grows to 9 nodes, message volume skyrockets because everyone messages everyone higher.
   - **Ring (Gold)**: Linear line $O(N)$. Message volume stays small and flat because the token simply circulates the circle twice.
2. **Election Duration Chart (Milliseconds to Elect)**:
   - Measures the exact elapsed time from King death to coronation. Bully completes faster because messages are sent in parallel; Ring is slightly slower because messages travel sequentially node-by-node.
3. **"Run Benchmark" Button**:
   - Automatically conducts 10 rounds of automated elections across different node counts, computes real statistical averages, and updates the charts live.
4. **Export CSV Button**:
   - Downloads a spreadsheet containing every election's duration and message count for your project report.

8. What the "Replay" Feature is Doing (Tab 3)

When you click **Replay**, you open a **Causal Time-Machine**.

> **The Fundamental Problem with Physical Time:**
> Independent computers have separate quartz hardware clocks that drift by milliseconds or seconds. You cannot trust physical timestamps to know which event happened first!

To solve this, Sovereign uses **Lamport Logical Clocks (Leslie Lamport, Turing Award 2013)**:

- Every node maintains a monotonically increasing integer counter (1, 2, 3...).
- Every network packet includes the sender's current clock.
- When receiving a message, a node updates its clock: `Clock = max(LocalClock, ReceivedClock) + 1`.
- This guarantees that if Event A caused Event B, then **Clock(A) < Clock(B)**.

Using the **Timeline Scrubber** in Tab 3, you can drag the slider back in time and step through history packet-by-packet to show examiners: *"At Lamport Tick 12 Node 5 died; at Tick 13 Node 1 noticed and called an election; at Tick 15 Node 4 replied OK; at Tick 18 Node 4 took the crown."*

9. Top 7 Viva Presentation Questions & Model Answers

Q1: What is the main objective of your project?

> "Sovereign is an interactive 3D distributed systems simulator. It runs real independent operating system processes communicating over TCP sockets, demonstrating how distributed nodes detect leader failure via periodic heartbeats and elect a new coordinator using the classic Bully and Ring algorithms."

Q2: Why is the Bully algorithm $O(N^2)$ while the Ring algorithm is $O(N)$?

> "In Bully, every node broadcasts election messages to all higher nodes in parallel. In the worst case, this requires $N(N-1)/2$ messages, which is $O(N^2)$. In Ring, an election token travels sequentially around the circle once, and a victory token travels around once, requiring approximately $2N$ messages, which is $O(N)$."

Q3: Why is it named the 'Bully' algorithm?

> "Because higher Node IDs have absolute authority over lower IDs. Any living higher-ID node can immediately bully lower nodes into submission by replying 'OK' and seizing control of the election."

Q4: How does your Ring algorithm handle node crashes along the circle?

> "Each node maintains the circular successor list. If its immediate neighbor's TCP socket is closed or unreachable, the node automatically catches the socket exception, skips the dead neighbor, and forwards the token to the next living successor in the ring."

Q5: Why did you implement Lamport Logical Clocks?

> "Physical clocks on different computers drift and cannot be synchronized perfectly. Lamport logical clocks assign an unambiguous causal sequence number to every event, enabling deterministic replay and auditing of the distributed execution."

Q6: What happens when the King crashes in the 3D visualization?

> "Followers detect the missing heartbeat within 2.2 seconds. The golden crown detaches from the dead castle, floats to the center sky above the island, and remains vacant until an election declares a new living sovereign."

Q7: Are Bully and Ring algorithms used in modern cloud databases like Google Spanner or AWS DynamoDB?

> "Rarely today, because neither handles network partitions (split-brain) well. Modern production cloud databases use quorum consensus algorithms like Raft or Multi-Paxos, which require majority agreement ((N/2)+1) before electing a leader."